AN AI POWERED SOFTWARE-BASED FIRE DETECTION SYSTEM FOR CCTV PLATFORMS.

FiremeX - TEST STRATEGY

Document ID	QA-01
Project	FiremeX
Prepared By	Jayashini Jayaweera
Date	-Jul-2026

1. Purpose

This Test Strategy defines the standard testing approach, methods, tools, and quality benchmarks applicable across all releases of the FiremeX, independent of any single release's Test Plan.

2. Testing Types & Approach

Testing Type	Approach
Functional Testing	Validate detection triggers, alert workflows, dashboard actions and configuration screens against requirements
UI Testing	Button clicks, form validations, text alignment, and cross-device compatibility.
Detection Accuracy Testing	Run curated video/image datasets (fire, smoke, false-positive triggers like sunlight/steam/red objects) through the model and measure precision/recall
Integration Testing	Validate camera integration, NVR integration, SMS/Email/Push gateway, and fire panel relay/API integration
Regression Testing	Automated smoke + regression suite executed on every build and every AI model retraining
Performance Testing	Simulate concurrent multi-camera streams (e.g., 4, 16, 64, 128 feeds) to measure detection latency and system stability
Security Testing	Video stream encryption, authentication/authorization, API penetration testing, RTSP stream hijack prevention
Usability Testing	Control-room dashboard clarity, alert visibility, alarm acknowledgment workflow
Reliability/Endurance Testing	24-72 hour continuous run to validate memory leaks, stream drops, and alert consistency
Compatibility Testing	Multiple camera brands, resolutions (720p/1080p/4K), and network conditions (LAN/VPN/4G backhaul)

3. Test Levels

- Unit Testing - done by developers on detection modules, API endpoints, notification services.
- Integration Testing - validates module-to-module and third-party system interfaces.
- System Testing - full end-to-end flow: camera feed > detection > alert > dashboard & Alert Section > alert acknowledgement > incident resolve
- Acceptance Testing - business/control-room users validate operational readiness.

4. Tools & Technology

Category	Tool(s)
Test Management	Kiwi TCMS
Functional Automation (UI)	Selenium WebDriver / Playwright
API Testing	Bruno / Insomnia

Category	Tool(s)
AI Model Accuracy Validation	ML Flow
Performance/Load Testing	Apache JMeter / Locust & custom RTSP stream simulator
Security Testing	Nmap
CI/CD Integration	Jenkins / GitHub Actions
Defect Tracking	OpenProject

5. Test Data Strategy

- Maintain a curated dataset : real fire, real smoke, near-miss (steam, fog, dust, red/orange objects, sunset glare), and normal scenes.
- Segregate datasets into Training-excluded validation sets to avoid data leakage into QA benchmarking.
- Refresh datasets quarterly and after every reported field false-positive/false-negative incident.

6. Defect Management

- Severity levels:
    1. Critical (missed fire detection / system down),
    2. High (delayed alert, false suppression),
    3. Medium (UI/reporting issue),
    4. Low (cosmetic).
- All Critical defects require root-cause analysis before closure.

7. Entry & Exit Philosophy

Every release must meet a minimum detection accuracy benchmark and pass the automated regression suite before promotion to user acceptance testing (UAT), regardless of feature scope, to protect the life-safety nature of the product.

8. Metrics Tracked

Metric	Target
True Positive Detection Rate	≥ 95%
False Positive Rate	≤ 2%
Alert Latency (detection to notification)	≤ 5 seconds
System Uptime during endurance test	≥ 99.5%
Test Case Pass Rate at release	≥ 95%
Critical/High Defect Escape Rate	0 to Production

9. Roles Involved in Strategy Governance

Quality Assurance member